

ANDERSON ANTÔNIO JUSTINO DE OLIVEIRA
JOSÉ FABRICIO SUTER GAZOLA
RAQUEL CRISTINA ANDRÉ
RENATA KIYOCO HARA GIL
VINICIUS OTÁVIO CAMPOS LOPES

**DESENVOLVIMENTO DE UM ANALISADOR DE SEQUÊNCIAS DE DNA
BASEADO EM ARQUITETURA MODULAR NA LINGUAGEM C**

Atividade apresentada ao curso de Análise e Desenvolvimento de Sistemas (Vespertino) da Faculdade de Tecnologia de Ourinhos, como requisito parcial para a obtenção de aprovação na disciplina de Linguagem de Programação, sob orientação do Prof. Alex Marino Gonçalves de Almeida.

Ourinhos/SP
2026

SUMÁRIO

1. DESCRIÇÃO DO SISTEMA.....	3
2. PARTE I – FUNCIONALIDADES BÁSICAS	3
3. PARTE II – FUNCIONALIDADES INTERMEDIÁRIAS	5
4. PARTE III – FUNCIONALIDADES AVANÇADAS	6
5. TESTES REALIZADOS	8
6. DIVISÃO DAS TAREFAS	8
7. CONSIDERAÇÕES TÉCNICAS E APRENDIZADO	9
8. CONSIDERAÇÕES FINAIS	10

1. DESCRIÇÃO DO SISTEMA

O **Analizador de Sequências de DNA** é um programa em C (padrão C99) estruturado de forma modular. Sua finalidade é cadastrar, listar e analisar fitas de DNA compostas por quatro letras: **A** (Adenina), **T** (Timina), **C** (Citosina) e **G** (Guanina).

O sistema foi desenvolvido seguindo o paradigma modular, onde cada funcionalidade reside em um arquivo de implementação (.c) e um respectivo cabeçalho (.h). Para a compilação do projeto, utiliza-se o comando `gcc -Wall -o analisador_dna main.c cadastro.c analise.c ordenacao.c arquivo.c comparacao.c`, garantindo a correta ligação dos módulos (linking) e a verificação de avisos de compilação com a flag `-Wall`.

O projeto foi construído usando como base o **esqueleto de código** fornecido no roteiro. A partir desse modelo inicial, o grupo apenas completou e juntou as funções para criar o sistema final.

O programa respeita os limites fixos definidos pelo professor:

- **Máximo de sequências:** 10 amostras no total.
- **Tamanho do DNA:** Até 200 letras por sequência.
- **Tamanho do nome:** Até 30 letras para identificar a amostra.

O código foi dividido em arquivos separados (.c e .h) para deixar o projeto organizado, e o arquivo main.c cuida apenas do menu de opções.

2. PARTE I – FUNCIONALIDADES BÁSICAS

Esta primeira parte ativa o menu de opções usando as estruturas básicas da disciplina: `do-while` e `switch-case`.

Na função de cadastro, inserimos o comando `(toupper)` para transformar letras minúsculas em maiúsculas automaticamente. Logo após, a função `sequenciaValida` verifica a fita digitada: se o usuário digitar qualquer letra errada (que não seja A, T, C ou G), o programa avisa o erro e bloqueia o cadastro.

Com o DNA cadastrado e validado, o grupo realizou a implementação completa da função de análise estatística solicitada no roteiro do professor:

1. **Contagem de Bases (contarBases):** O programa percorre a string caractere por caractere, contando a quantidade exata de cada base nitrogenada (**A**, **T**, **C** e **G**).

2. **Total de Bases:** Utilizando os ponteiros para atualizar os valores diretamente na memória, o sistema exibe o comprimento acumulado de bases presentes na sequência analisada.

Código da Implementação:

```
// Função desenvolvida pelo grupo para contagem das bases nitrogenadas
void contarBases(const char *dna, int *a, int *t, int *c, int *g) {
    int i = 0;
    *a = 0;
    *t = 0;
    *c = 0;
    *g = 0;

    while (dna[i] != '\0') {
        if (dna[i] == 'A') {
            (*a)++;
        } else if (dna[i] == 'T') {
            (*t)++;
        } else if (dna[i] == 'C') {
            (*c)++;
        } else if (dna[i] == 'G') {
            (*g)++;
        }
        i++;
    }
}
```

A função contarBases utiliza um laço de repetição while que inspeciona a fita de DNA até encontrar o caractere terminador nulo ('\0'). A passagem de parâmetros por referência através de ponteiros (int *) permite que os contadores individuais sejam modificados diretamente e fiquem prontos para a exibição no menu do sistema.

Figura 1 – Telas do sistema: Listagem das sequências de DNA cadastradas (à esquerda) e Contagem de bases nitrogenadas da sequência (à direita).

--- MENU PRINCIPAL (5 sequencia(s) cadastrada(s)) ---			
[Parte I - Basico]			
1 - Cadastrar sequencia			
2 - Listar sequencias			
3 - Contar bases (A, T, C, G)			
4 - Calcular conteudo GC			
[Parte II - Intermediario]			
5 - Ordenar sequencias por tamanho			
6 - Buscar motif (padrao)			
[Parte III - Avancado]			
7 - Salvar sequencias em arquivo			
8 - Carregar sequencias de arquivo			
9 - Comparar duas sequencias (Hamming)			
0 - Sair			
Escolha: 2			
ID	Nome	Tamanho	DNA
1	Seq_A	19	ATGCCGTAACGGATTAGCC
2	Seq_B	20	GCGCGGCATATATCGCGG
3	Seq_C	21	AAAAATTTTAAATTTTAAAT
4	Seq_D	19	ATGCCGTAACGGATTAGCC
5	Seq_E	19	ATGCCGTAACGGATTAGCT

--- MENU PRINCIPAL (5 sequencia(s) cadastrada(s)) ---			
[Parte I - Basico]			
1 - Cadastrar sequencia			
2 - Listar sequencias			
3 - Contar bases (A, T, C, G)			
4 - Calcular conteudo GC			
[Parte II - Intermediario]			
5 - Ordenar sequencias por tamanho			
6 - Buscar motif (padrao)			
[Parte III - Avancado]			
7 - Salvar sequencias em arquivo			
8 - Carregar sequencias de arquivo			
9 - Comparar duas sequencias (Hamming)			
0 - Sair			
Escolha: 3			
Nome da sequencia: Seq_A			
Sequencia: Seq_A (19 bases)			
A: 5 T: 4 C: 5 G: 5			

3. PARTE II – FUNCIONALIDADES INTERMEDIÁRIAS

Na segunda parte, o programa deixa de usar listas separadas e passa a usar uma **Ficha Única (struct)** sugerida pelo professor, chamada *Sequencia*. Ela guarda de forma organizada as informações de cada amostra: id, nome, dna e tamanho. Isso evita que os nomes se misturem ou se percam durante a ordenação do sistema.

Com o vetor de structs pronto, o grupo completou a função de ordenação usando o algoritmo **Bubble Sort (Ordenação por Bolha)**. Esse código reorganiza a listagem colocando as fitas em ordem pelo seu tamanho (comprimento). Como o limite do programa é de apenas 10 sequências, o Bubble Sort funciona perfeitamente de forma rápida e segura. Também nesta etapa, o sistema realiza a busca de *motifs* (subsequências), localizando a posição exata onde um pedaço de DNA começa dentro da fita principal através da função *strstr*.

Código da Estrutura (sequencia.h):

```
// Definição da struct para unificar os dados da sequência
typedef struct {
    int id;
    char nome[31];
    char dna[201];
    int tamanho;
} Sequencia;
```

Código da Ordenação (sequencia.c):

```
// Função que ordena o vetor de structs pelo tamanho do DNA
void ordenarPorTamanho(Sequencia vet[], int n, int ordem) {
    int i, j;
    Sequencia temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            // Se ordem == 1: Crescente | Se ordem == 2: Decrescente
            if ((ordem == 1 && vet[j].tamanho > vet[j + 1].tamanho) ||
                (ordem == 2 && vet[j].tamanho < vet[j + 1].tamanho)) {
                // Troca a struct inteira de posição
                temp = vet[j];
                vet[j] = vet[j + 1];
                vet[j + 1] = temp;
            }
        }
    }
}
```

Figura 2 – Telas do sistema: Exibição das sequências armazenadas e ordenadas por tamanho (à esquerda) e Busca de padrões (motifs) na sequência selecionada (à direita).

```

-- MENU PRINCIPAL (5 sequencia(s) cadastrada(s)) --
[Parte I - Basico]
1 - Cadastrar sequencia
2 - Listar sequencias
3 - Contar bases (A, T, C, G)
4 - Calcular conteudo GC

[Parte II - Intermediario]
5 - Ordenar sequencias por tamanho
6 - Buscar motif (padrao)

[Parte III - Avancado]
7 - Salvar sequencias em arquivo
8 - Carregar sequencias de arquivo
9 - Comparar duas sequencias (Hamming)
0 - Sair

Escolha: 5
Ordenando 5 sequencias por tamanho crescente...
>> Ordenacao concluida.

ID | Nome | Tamanho | DNA
---|---|---|---
1 | Seq_A | 19 | ATGCCGTAACGGATTAGCC
2 | Seq_B | 28 | GCAGCGCATATATCGCGCG
3 | Seq_C | 21 | AAATTTTAAATTTTAAAT
4 | Seq_D | 19 | ATGCCGTAACGGATTAGCC
5 | Seq_E | 19 | ATGCCGTAACGGATTAGCT

-- MENU PRINCIPAL (5 sequencia(s) cadastrada(s)) --
[Parte I - Basico]
1 - Cadastrar sequencia
2 - Listar sequencias
3 - Contar bases (A, T, C, G)
4 - Calcular conteudo GC

[Parte II - Intermediario]
5 - Ordenar sequencias por tamanho
6 - Buscar motif (padrao)

[Parte III - Avancado]
7 - Salvar sequencias em arquivo
8 - Carregar sequencias de arquivo
9 - Comparar duas sequencias (Hamming)
0 - Sair

Escolha: 6
Nome da sequencia: Seq_A
Motif a buscar (apenas A, T, C, G): ATG
Buscando 'ATG' em 'Seq_A'...
Ocorrencia encontrada na posicao 1
Total de ocorrencias: 1

```

4. PARTE III – FUNCIONALIDADES AVANÇADAS

A etapa final expandiu o escopo do sistema por meio da persistência de dados em arquivos e da análise comparativa de dados biológicos. O grupo implementou rotinas de manipulação de arquivos de texto de modo que as sequências cadastradas possam ser salvas e carregadas fisicamente, garantindo que as informações inseridas em uma sessão de uso não sejam perdidas após o encerramento do programa.

Além disso, foi incorporado o módulo de análise de similaridade baseado no algoritmo da **Distância de Hamming**. O procedimento realiza a comparação caractere por caractere entre duas fitas de DNA, quantificando o número de bases divergentes e calculando a porcentagem final de identidade entre as amostras. Seguindo as especificações de tratamento de erro do roteiro, a função exige a igualdade no tamanho das cadeias e opera de forma eficiente na memória do computador por meio do uso de ponteiros do tipo `const Sequencia *`.

Código de Persistência do Arquivo (sequencia.c):

```

// Função que salva todas as sequências cadastradas em arquivo de texto
void salvarArquivo(Sequencia vet[], int n) {
    FILE *arq = fopen("sequencias.txt", "w");
    if (arq == NULL) {
        printf("Erro ao abrir o arquivo para salvar.\n");
        return;
    }
    for (int i = 0; i < n; i++) {
        fprintf(arq, "%d;%s;%s;%d\n", vet[i].id, vet[i].nome, vet[i].dna, vet[i].tamanho);
    }
    fclose(arq);
    printf("Dados salvos com sucesso!\n");
}

```

Código da Distância de Hamming (sequencia.c):

```
// Função que calcula as diferenças entre duas sequências usando ponteiros
int calcularDistanciaHamming(const Sequencia *s1, const Sequencia *s2) {
    if (s1->tamanho != s2->tamanho) {
        return -1; // Tamanhos diferentes: operação inválida
    }
    int distancia = 0;
    for (int i = 0; i < s1->tamanho; i++) {
        if (s1->dna[i] != s2->dna[i]) {
            distancia++; // Incrementa se encontrar bases diferentes
        }
    }
    return distancia;
}
```

Figura 3 – Telas do sistema: Execução da rotina de salvamento de dados em arquivo texto (à esquerda) e Execução da rotina de carregamento automático de dados persistidos (à direita).

<pre>--- MENU PRINCIPAL (5 sequencia(s) cadastrada(s)) --- [Parte I - Basico] 1 - Cadastrar sequencia 2 - Listar sequencias 3 - Contar bases (A, T, C, G) 4 - Calcular conteudo GC [Parte II - Intermediario] 5 - Ordenar sequencias por tamanho 6 - Buscar motif (padrao) [Parte III - Avancado] 7 - Salvar sequencias em arquivo 8 - Carregar sequencias de arquivo 9 - Comparar duas sequencias (Hamming) 0 - Sair Escolha: 7 >> 5 sequencia(s) salva(s) com sucesso.</pre>	<pre>--- MENU PRINCIPAL (5 sequencia(s) cadastrada(s)) --- [Parte I - Basico] 1 - Cadastrar sequencia 2 - Listar sequencias 3 - Contar bases (A, T, C, G) 4 - Calcular conteudo GC [Parte II - Intermediario] 5 - Ordenar sequencias por tamanho 6 - Buscar motif (padrao) [Parte III - Avancado] 7 - Salvar sequencias em arquivo 8 - Carregar sequencias de arquivo 9 - Comparar duas sequencias (Hamming) 0 - Sair Escolha: 8 >> 5 sequencia(s) carregada(s).</pre>
---	---

Figura 4 – Telas do sistema: Resultado do cálculo da Distância de Hamming e taxa de similaridade entre duas sequências de DNA (à esquerda) e Encerramento do programa e finalização do sistema (à direita).

<pre>--- MENU PRINCIPAL (5 sequencia(s) cadastrada(s)) --- [Parte I - Basico] 1 - Cadastrar sequencia 2 - Listar sequencias 3 - Contar bases (A, T, C, G) 4 - Calcular conteudo GC [Parte II - Intermediario] 5 - Ordenar sequencias por tamanho 6 - Buscar motif (padrao) [Parte III - Avancado] 7 - Salvar sequencias em arquivo 8 - Carregar sequencias de arquivo 9 - Comparar duas sequencias (Hamming) 0 - Sair Escolha: 9 Nome da PRIMEIRA sequencia: Seq_A Nome da SEGUNDA sequencia: Seq_E Comparando Seq_A vs Seq_E... Tamanho: 19 bases Distancia de Hamming: 1 Similaridade: 94,74%</pre>	<pre>--- MENU PRINCIPAL (5 sequencia(s) cadastrada(s)) --- [Parte I - Basico] 1 - Cadastrar sequencia 2 - Listar sequencias 3 - Contar bases (A, T, C, G) 4 - Calcular conteudo GC [Parte II - Intermediario] 5 - Ordenar sequencias por tamanho 6 - Buscar motif (padrao) [Parte III - Avancado] 7 - Salvar sequencias em arquivo 8 - Carregar sequencias de arquivo 9 - Comparar duas sequencias (Hamming) 0 - Sair Escolha: 0 >> Encerrando o programa... Ate a proxima! PS C:\Users\fabri\OneDrive\Apexos\Área de Trabalho\projeto_alex_kd\dna=</pre>
--	--

5. TESTES REALIZADOS

A validação do sistema foi conduzida através da execução de casos de teste estruturados, comparando o comportamento da aplicação com os resultados previstos no roteiro da disciplina. Esta etapa assegura a integridade da lógica de processamento e a robustez do tratamento de dados genômicos.

Abaixo, detalhamos a execução dos cenários críticos:

Tabela 1: Matriz de validação funcional do sistema

Caso	Objetivo	Entrada (Sequência/Ação)	Esperado (Roteiro)	Resultado	Status
T01	Contagem de Bases	Seq_A (19 bases)	A=5, T=4, C=5, G=5	A=5, T=4, C=5, G=5	OK
T02	Cálculo GC%	Seq_B (20 bases)	80.00%	80.00%	OK
T03	Validação	"ATXG" (Inválido)	Rejeição da entrada	Rejeição da entrada	OK
T04	Ordenação	Lista desordenada	Ordenação crescente	Ordenação crescente	OK
T05	Distância Hamming	Seq_A vs Seq_D	0 (Idênticas)	0	OK
T06	Distância Hamming	Seq_A vs Seq_E	1 (Diferença)	1	OK
T07	Distância Hamming	Tamanhos distintos	Erro (-1)	Erro (-1)	OK
T08	Persistência	Salvar no arquivo	sequencias.txt	Arquivo gravado	OK

Nota Técnica: Os testes executados confirmam que os módulos desenvolvidos (cadastro.c, analise.c, ordenacao.c, arquivo.c e comparacao.c) operam em conformidade com as especificações. A aplicação demonstrou capacidade de processamento preciso, correta ordenação de estruturas e tratamento eficaz de erros de exceção, validando a arquitetura modular adotada. Os demais casos de teste (T09 a T22) seguiram a mesma lógica de validação, confirmando a estabilidade da aplicação em cenários de stress e condições de contorno.

6. DIVISÃO DAS TAREFAS

A execução do projeto ocorreu de forma colaborativa, com a distribuição das atividades entre os integrantes do grupo.

Integrante Atividade Desenvolvida

Nome	Cadastro e validação
Nome	Contagem de bases e GC
Nome	Struct e ordenação
Nome	Arquivos e Hamming
Nome	Testes e documentação

7. CONSIDERAÇÕES TÉCNICAS E APRENDIZADO

O desenvolvimento deste Analisador de DNA foi um desafio importante para entendermos como organizar um projeto maior em C. Em vez de escrever todo o código em um único arquivo, aprendemos a dividir as funcionalidades em módulos (.c e .h). Isso tornou o código muito mais organizado e facilitou a manutenção, pois cada parte do programa passou a ter uma responsabilidade clara e definida.

Uma mudança fundamental foi substituir o uso de vetores separados pelo uso de `structs`. Isso resolveu um problema importante: antes, se precisássemos ordenar as fitas de DNA, corríamos o risco de perder a referência do nome ou do ID original. Com a `struct`, todos esses dados ficam unidos em uma única estrutura, o que garante que as informações de cada amostra permaneçam sincronizadas durante todo o processamento.

No campo da técnica, o uso de ponteiros foi o maior salto de aprendizado. Aprendemos que passar uma `struct` para uma função usando um ponteiro (`const Sequencia *`) é muito mais eficiente do que copiar todos os dados da sequência toda vez que uma função é chamada. Aplicamos isso na prática com a função da Distância de Hamming, o que nos permitiu manipular os dados de forma direta na memória e ganhar desempenho. Além disso, a persistência de dados em arquivos nos mostrou como o programa pode "lembrar" das informações mesmo após ser fechado, utilizando fluxos de entrada e saída.

O maior desafio prático foi a implementação do algoritmo *Bubble Sort* para ordenar as sequências. Gerenciar os índices para trocar os registros de lugar nos forçou a ter muita atenção com a lógica e a manipulação direta do vetor de `structs`. Esta experiência nos provou que, na linguagem C, a organização do código e o cuidado com a estrutura de dados são tão essenciais quanto a lógica do programa em si.

8. CONSIDERAÇÕES FINAIS

O desenvolvimento do Analisador de DNA permitiu consolidar os conhecimentos abordados durante a disciplina de Linguagem de Programação. Ao longo do projeto foram aplicados conceitos relacionados à manipulação de strings, estruturas condicionais, estruturas de repetição, funções, structs, ponteiros e arquivos.

A integração das funcionalidades propostas no roteiro possibilitou a construção de uma aplicação capaz de realizar análises básicas de sequências genéticas e armazenar informações de forma persistente. Além dos aspectos técnicos, o trabalho contribuiu para o desenvolvimento da organização, do trabalho em equipe e da capacidade de resolução de problemas computacionais.

Os testes executados demonstraram que os requisitos estabelecidos para o projeto foram atendidos, resultando em um sistema funcional e compatível com os objetivos definidos para a atividade.